5.2 Objects, arrays, and loops

Twig can receive objects and arrays as well as strings. We will prepare the data in PHP and let the template display it.

Add a model

Create `src/Tutorials/Class5/Product.php`:

```
<?php
declare(strict_types=1);

namespace App\Tutorials\Class5;

class Product
{
    public function __construct(
        private string $description,
        private float $price,
    ) {
    }

    public function getDescription(): string
    {
        return $this->description;
    }

    public function getPrice(): float
    {
        return $this->price;
    }

    public function __toString(): string
    {
        return $this->description . ' ($' . number_format($this->price, 2) . ')';
    }
}
```

Pass several kinds of data

Replace only `run()` in `src/Tutorials/Class5/Application.php` with this method. Keep the namespace, imports, property, constant, and constructor from 5.1. Product is in the same namespace, so it needs no additional import.

```
public function run(): void
{
    $products = [
```

```
            new Product('Club notebook', 4.50),
            new Product('Club mug', 8.00),
        ];
        $template = 'demo.html.twig';
        $args = [
            'name' => 'Avery',
            'meeting' => ['day' => 'Wednesday', 'room' => 'Library 204'],
            'product' => $products[0],
            'products' => $products,
        ];
        echo $this->twig->render($template, $args);
    }
```

Create `templates/tutorials/class5/demo.html.twig`:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Club data</title>
</head>
<body>
    <h1>Hello {{ name }}</h1>
    <p>Meet {{ meeting.day }} in {{ meeting.room }}.</p>
    <p>Featured: {{ product.getDescription() }} — ${{
product.price|number_format(2) }}</p>
    <p>Summary: {{ product }}</p>
    <ul>
        {% for item in products %}
            <li>{{ item.description }} — ${{ item.price|number_format(2) }}</li>
        {% else %}
            <li>No products available.</li>
        {% endfor %}
    </ul>
</body>
</html>
```

Explain the notation

Twig	Meaning in this example
`meeting.room`	Read the room key of the array
`product.getDescription()`	Call the public getter explicitly
`product.price`	Resolve through public getPrice(); it does not bypass private visibility
`product` in output braces	Convert the object to a string using __toString()
`price	number_format(2)`	

Twig	Meaning in this example
`{% for item in products %}`	Repeat markup for each item

`{{ ... }}` outputs a value; `{% ... %}` performs a template instruction. A Twig comment uses `{# ... #}` and is not included in the generated HTML. The loop's `else` runs when there are no items. The pipe applies a filter; it is not PHP's object operator.

Expect the meeting room, the featured notebook priced at $4.50, its string summary, and two list items. To test an empty list, change only the context entry to `'products' => []`; keep the featured product intact. Expect “No products available.” Restore the original entry before continuing.

Reference: Twig template syntax.